The National Higher School of Artificial Intelligence
المدرسة الوطنية العليا للذكاء الاصطناعي

Course
« *Computer Vision*»

Sid-Ahmed Berrani

2025-2026

Spatial filtering

It is a neighborhood operator

It uses a collection of pixel values in the vicinity of a given pixel to determine its final output value.

In a linear spatial filter, the output pixel value is a weighted sum of pixel values within a neighborhood *N*.

Each set of weights => define a new filter

It is a **spatial filtering**.

The underlying mechanism is the ***convolution***.

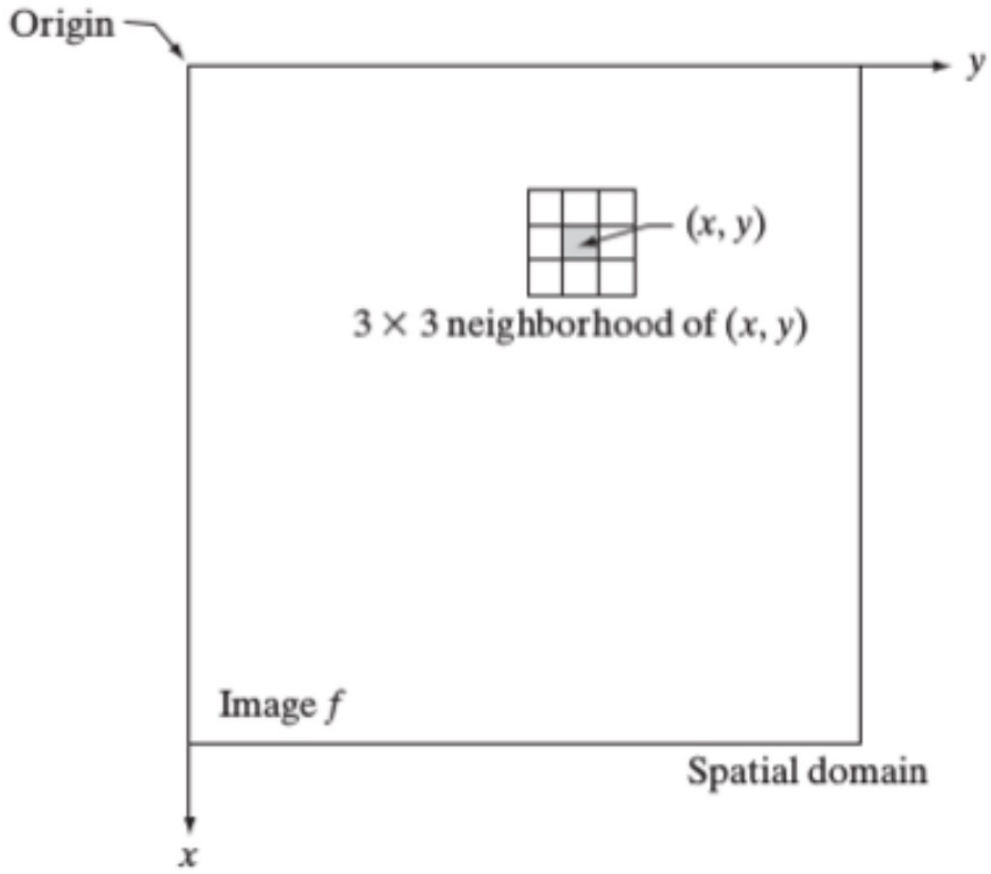
Spatial filtering

It creates a new pixel with coordinates equal to the center of the neighborhood and whose value is the result of the filtering operation.

Filter is defined in terms of a coefficient matrix W

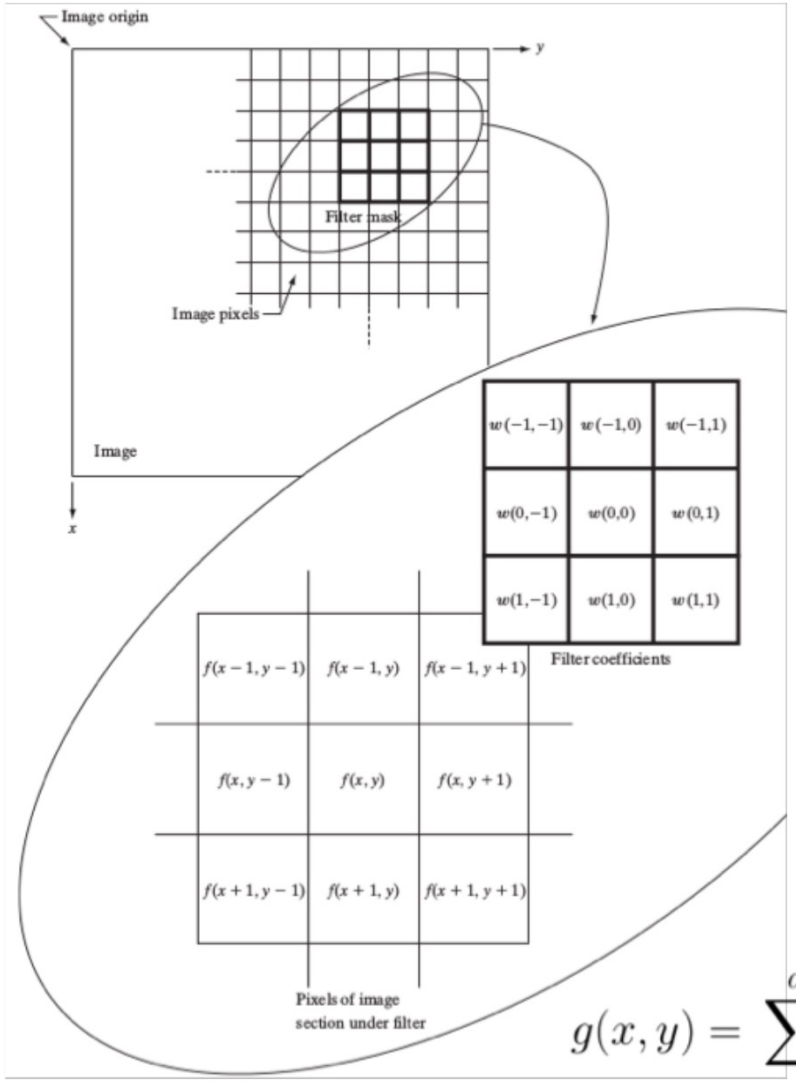
The performed operation is the sum of products of the filter coefficients and the image pixels encompassed by the filter.

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$



Spatial filtering

- When applying a linear filter, the same set of weights is applied on the whole image.
- Different sets of weights => different processes.



Example of a filter acting on a 3x3 neighborhood:

$$g(x,y) = w(-1,-1)f(x-1,y-1)+ \\ +w(-1,0)f(x-1,y)+ \quad \cdots \\ \cdots \\ +w(1,1)f(x+1,y+1)$$

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)f(x+s,y+t)$$

Spatial filtering

An example: The average filter

A linear filter that averages all pixels within *2k+1 x 2k+1* neighborhood:

$$R_{ij} = \frac{1}{(2k+1)^2} \sum_{u=i-k}^{i+k} \sum_{v=j-k}^{j+k} f_{uv}$$

$\frac{1}{9} \times$

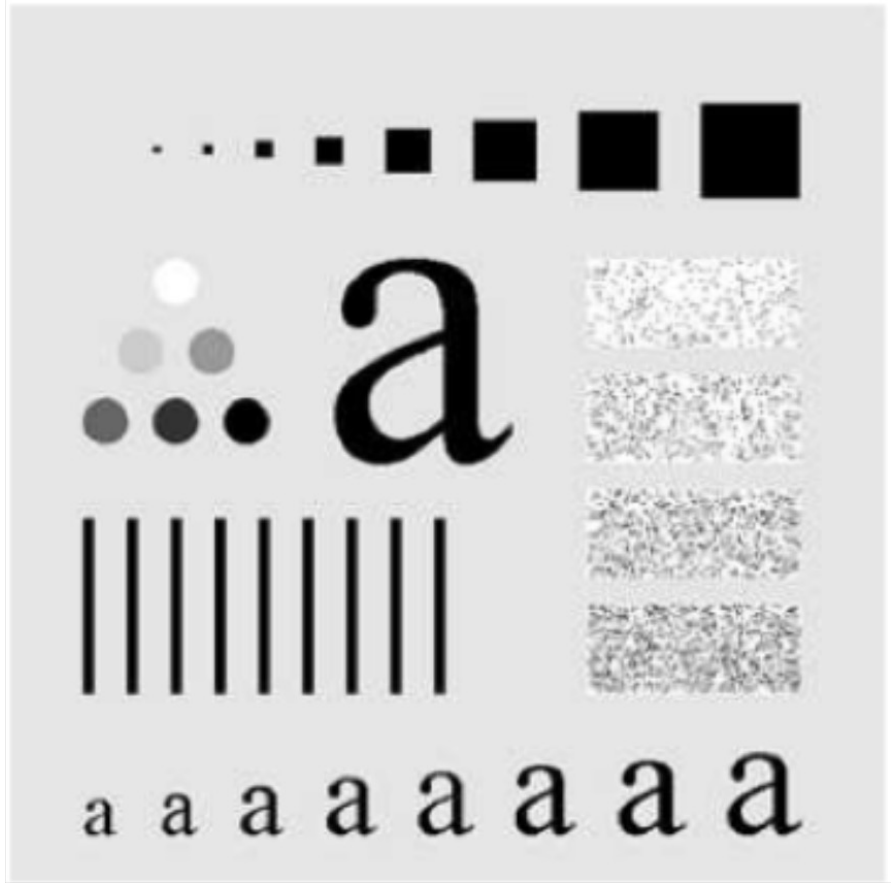
1	1	1
1	1	1
1	1	1

The filter (kernel) is a matrix of size 2k+1 x 2k+1 where all the components of the filters are equal to 1.

Spatial filtering

An example: The average filter

Original image

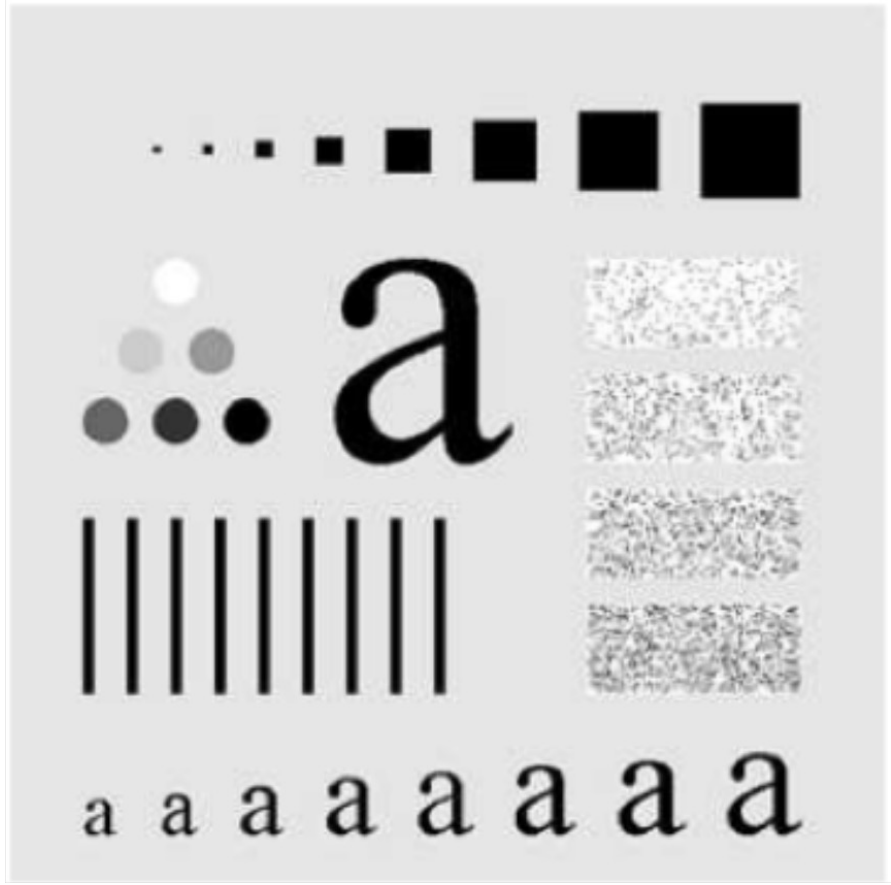


3x3 average filter

Spatial filtering

An example: The average filter

Original image

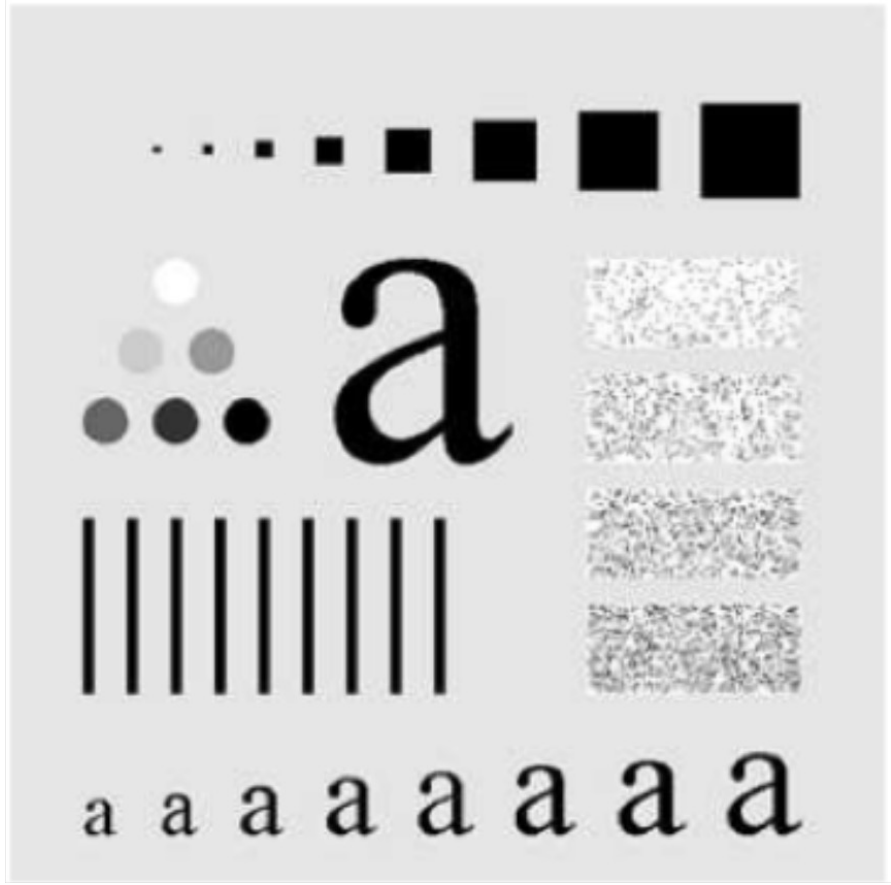


5x5 average filter

Spatial filtering

An example: The average filter

Original image



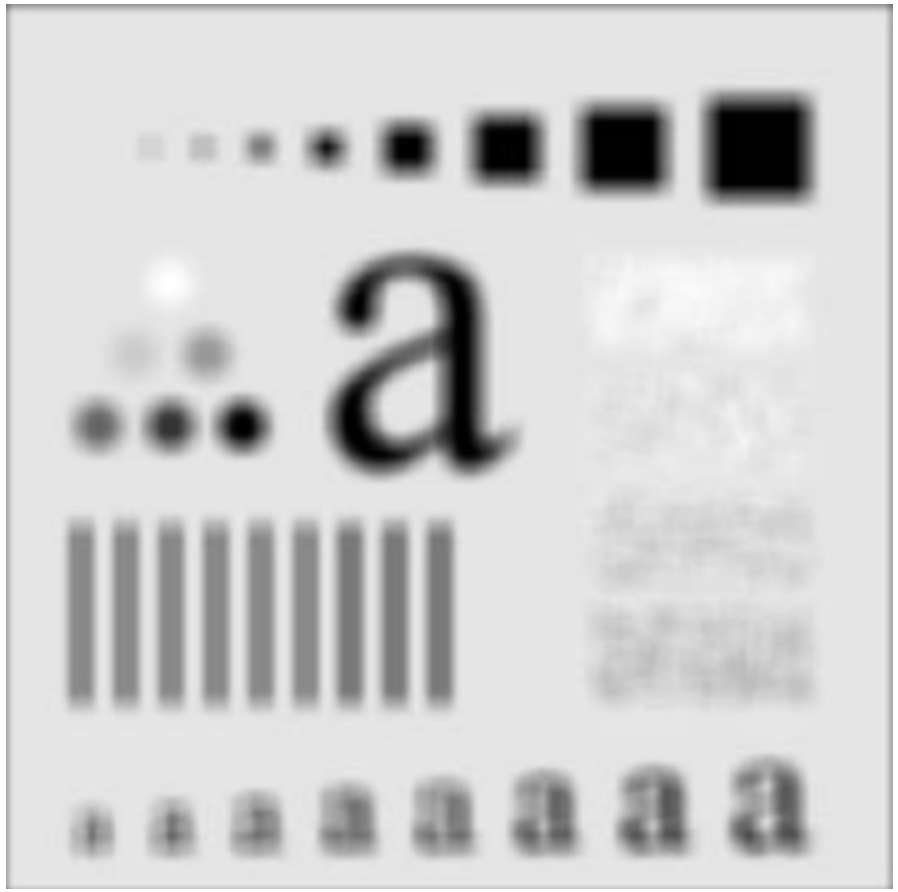
9x9 average filter

Spatial filtering

An example: The average filter

Original image

15x15 average filter



Spatial filtering

Properties of filters, in general

- **Superposition** (additivity): $R(f + g) = R(f) + R(g)$.

  Filtering the sum of two images is the same as filtering them separately and adding the results.

- **Scaling** (homogeneity): $R(kf) = kR(f)$.

  If we multiply the image intensity by a factor, the filtered result is multiplied by the same factor.

- **Linear:** Superposition + scaling.

  A filter is linear if it satisfies superposition (additivity) and scaling

Spatial filtering

Properties of filters, in general

- **Shift invariance:** The response to a *translated* stimulus is just a translation of the response to the stimulus.

$$R\big(f(x - x_0)\big) = R(f)(x - x_0)$$

Same behavior everywhere in the image.

If the image shifts, the filtered result shifts in the same way.

⇒ **A filter that is linear and shift-invariant can be implemented as a convolution.**

Spatial linear filtering

What it is it used for?

- **Noise Reduction –** Removes unwanted noise while preserving essential image features.
- **Edge Detection –** Enhances edges by applying filters like Sobel, Prewitt, or Laplacian.
- **Image Sharpening –** Enhances fine details and textures by emphasizing high-frequency components.
- **Smoothing/Blurring –** Reduces details or noise using averaging or Gaussian filters.
- **Feature Enhancement –** Enhances specific patterns or structures, such as textures.
- **Embossing and Edge Enhancement –** Creates relief effects by highlighting directional intensity changes.
- **Image Restoration –** Improves image quality by removing artifacts or blurring effects.

Smoothing filters

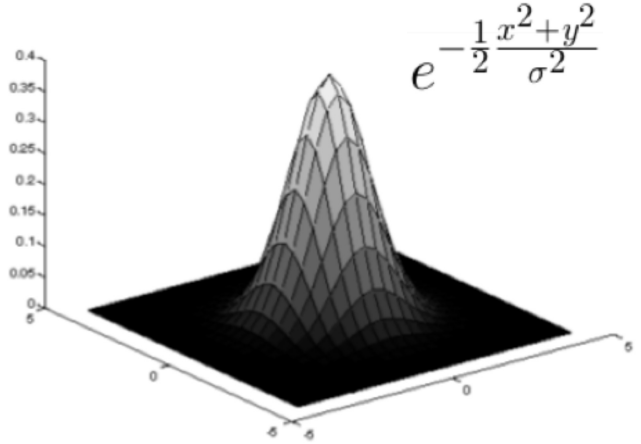
Noise appears as rapid intensity variations.

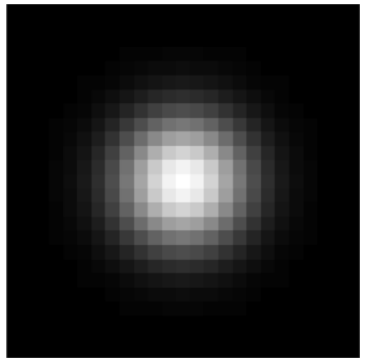
=> Smoothing filters reduce noise by averaging neighboring pixels.

The **average filter** is an example.

The **Gaussian filter** is another smoothing filter:

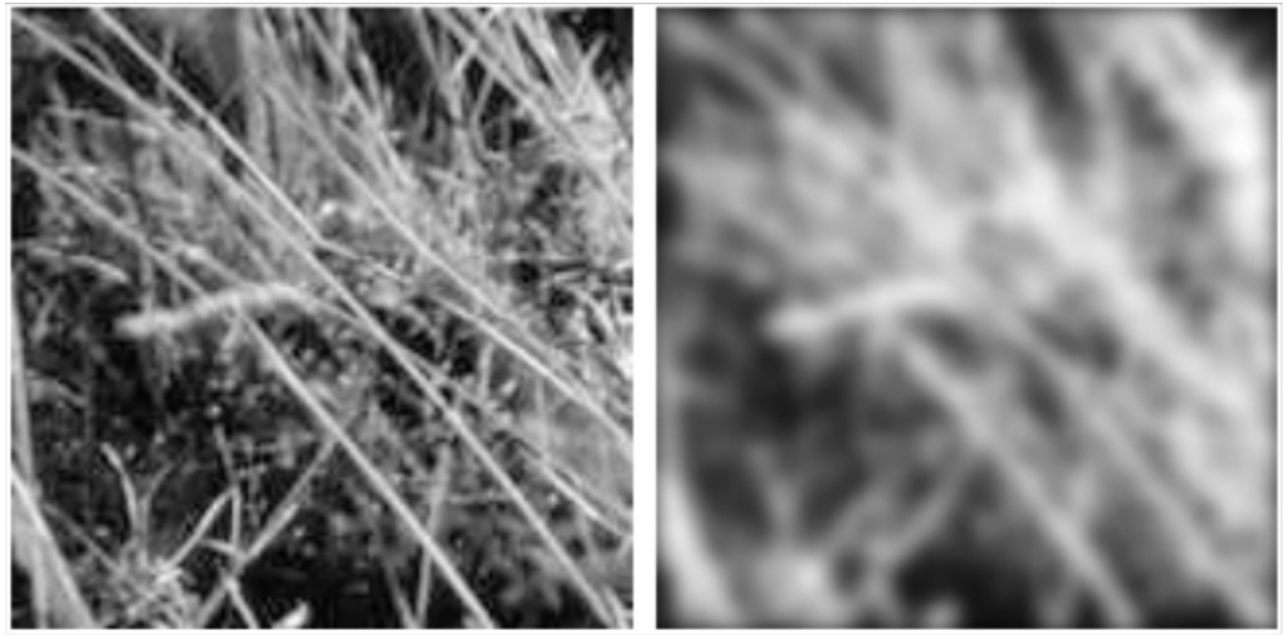
- It is a weighted average filter where closer pixels matter more.
- Gaussian smoothing is a linear shift-invariant filter
  → convolution.





Smoothing filters

$$e^{-\frac{1}{2}\frac{x^2+y^2}{\sigma^2}}$$



The **Gaussian filter** => A crucial parameter: σ

- σ controls the amount of blur:

  Small σ → little smoothing

  Large σ → strong smoothing

Smoothing filters

The **Gaussian filter**:

- Small σ → only very close pixels influence the average → little blur
- Large σ → many neighbors influence the average → strong blur
- Very large σ → image details disappear

σ determines the size of the neighborhood used for averaging.

- In practice, the Gaussian filter is implemented as a convolution mask.
- A discrete smoothing kernel *H* is obtained by constructing a *2k+1 x 2k+1* array whose *i, j*th value is:

$$H_{ij} = \frac{1}{2\pi\sigma^2} exp\left(-\frac{\left((i-k-1)^2+(j-k-1)^2\right)}{2\sigma^2}\right)$$

Smoothing filters

Why Gaussian instead of average filter?”

Key points:

- Average filter treats all neighbors equally → causes artifacts*
- Gaussian gives more weight to closer pixels → smoother and more natural results

**Artifacts are unwanted visual distortions or errors that appear in an image as a result of processing, compression, or acquisition.*

Sharpening filters

The principal objective of sharpening is to highlight transitions in intensity.

Sharpening enhances edges by amplifying rapid intensity changes.

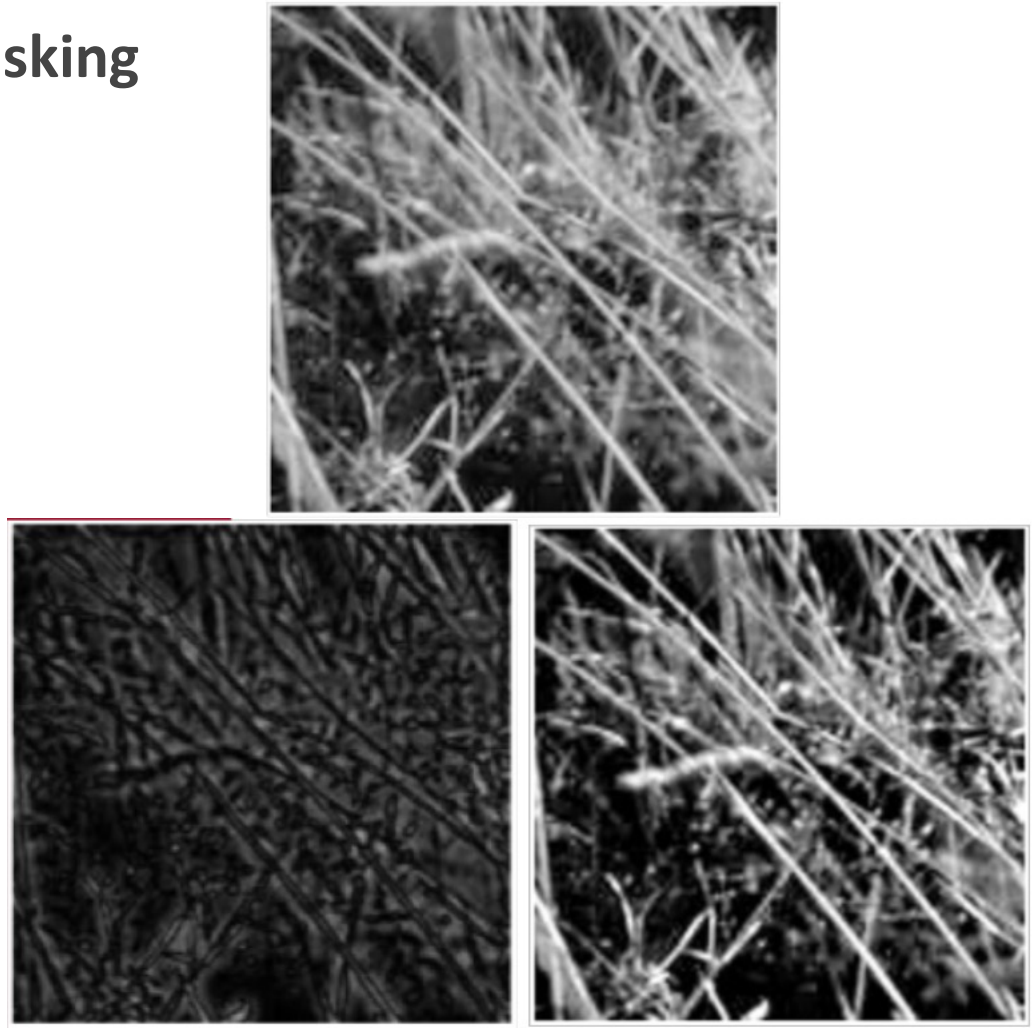
Sharpening filters

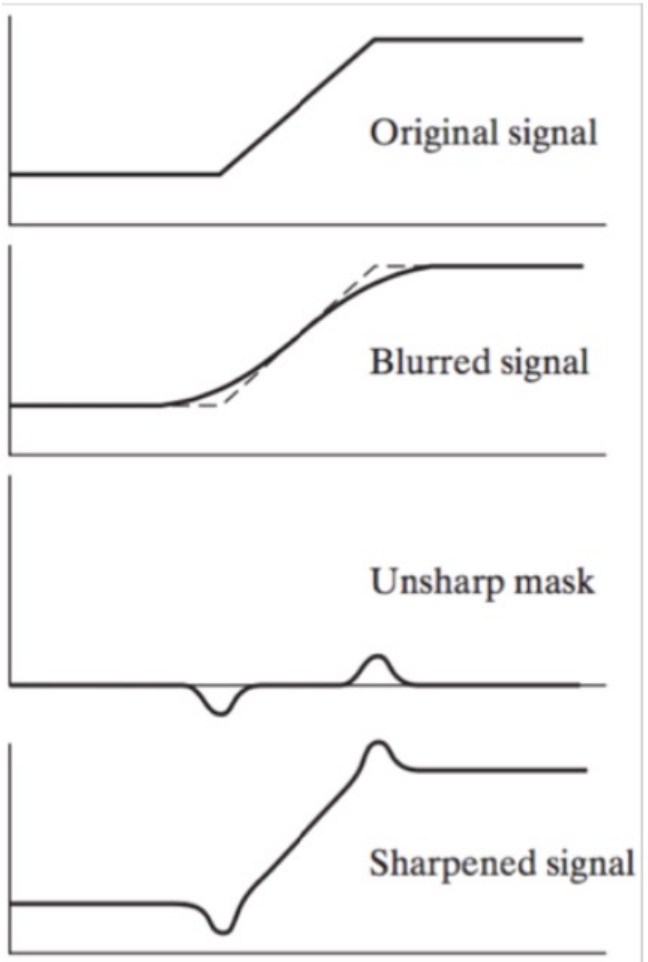
One simple approach is called **unsharp masking** (or high-boost filtering) and consist of the following:

1. Blur the original image
2. Subtract the blurred image from the original to obtain a mask
3. Add the mask to the original (multiplied by a constant for high-boosting)

Sharpening filters

Unsharp masking





Sharpening filters

In general, sharpening filter are based on the concept of differentiation.

Image blurring	Image sharpening
Averaging pixels in a neighborhood: Integration	Differentiation (first or second order derivatives)
Remove details	Enhance details

Sharpening filters

Derivatives of a digital discrete functions are defined in term of differences.

First-order derivative of a one-dimensional function:

$$\frac{df}{dx} = \lim_{\epsilon \to 0} \frac{f(x+\epsilon) - f(x)}{\epsilon} \approx f(x+1) - f(x)$$

Second-order derivative:

$$\frac{d^2 f}{d^2 x} \approx f(x+1) + f(x-1) - 2f(x)$$

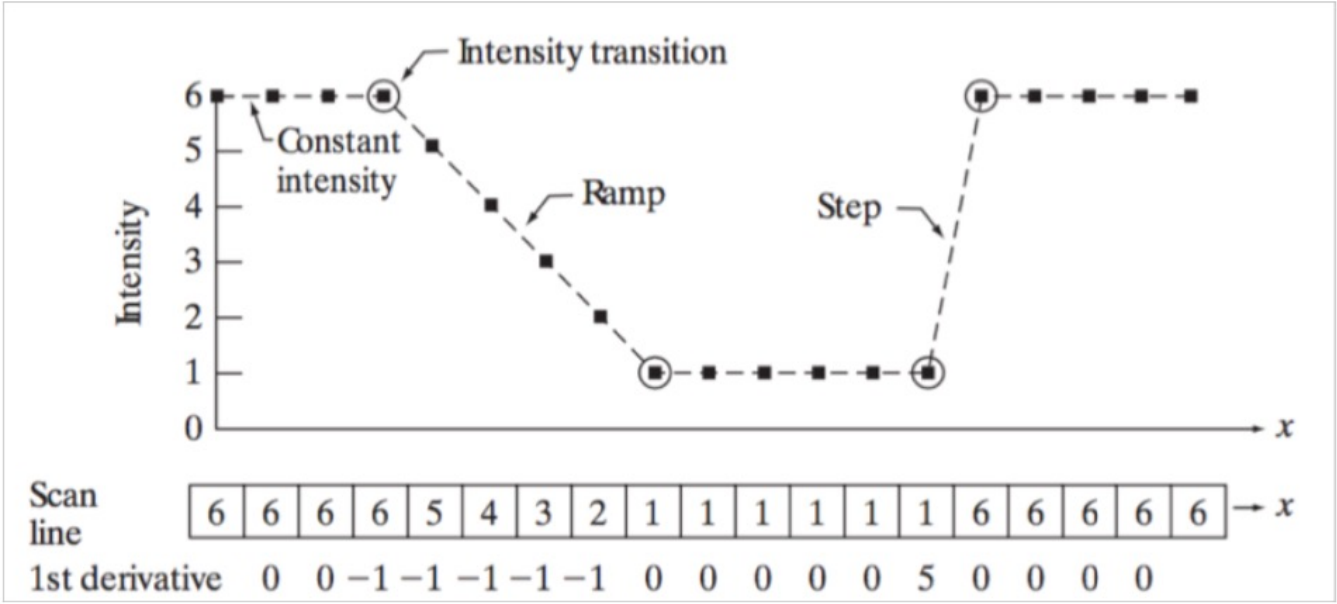
Sharpening filters

Derivatives of a digital discrete functions are defined in term of differences.

First-order derivative of a one-dimensional function: $\frac{df}{dx} = \lim_{\epsilon \to 0} \frac{f(x+\epsilon) - f(x)}{\epsilon} \approx f(x+1) - f(x)$

Derivatives measure how fast intensity changes.

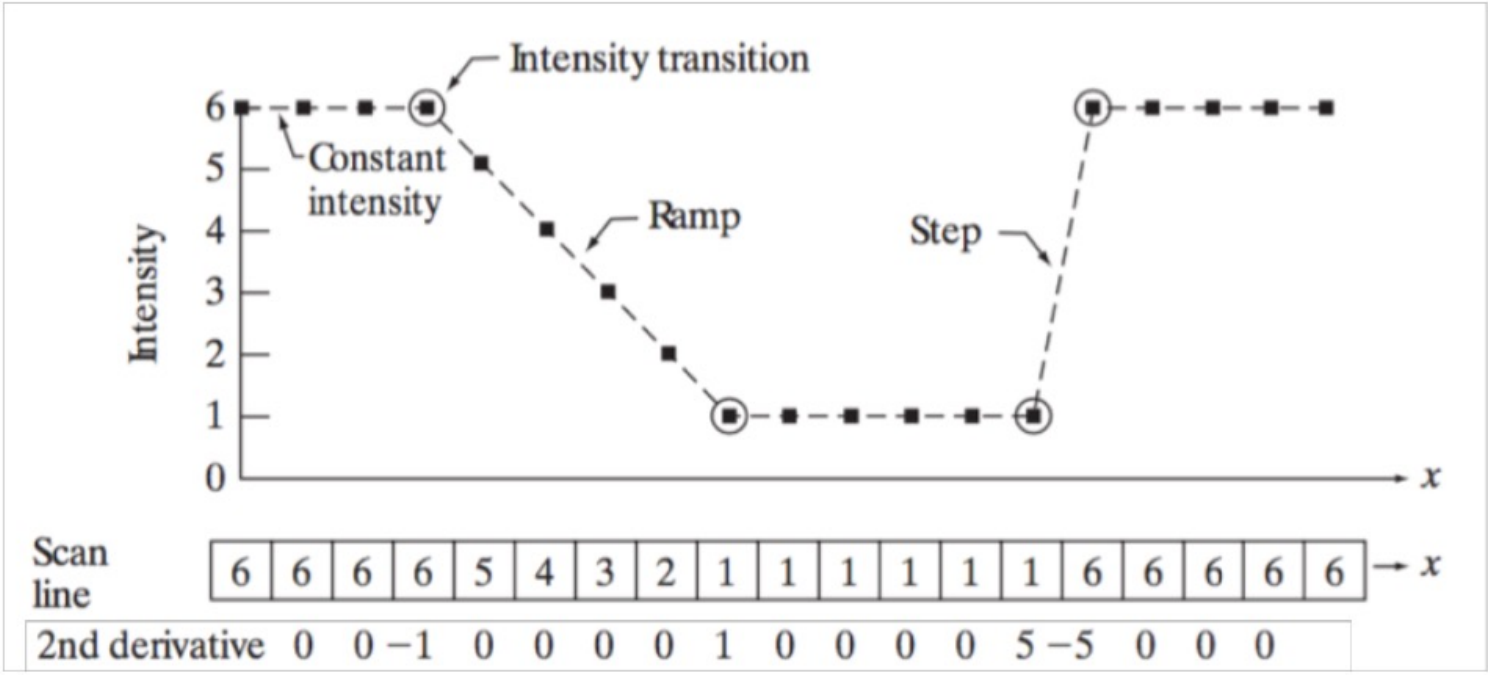
Small change → smooth region
Large change → edge



Sharpening filters

Derivatives of a digital discrete functions are defined in term of differences.

Second-order derivative: $\frac{d^2 f}{d^2 x} \approx f(x+1) + f(x-1) - 2f(x)$



Sharpening filters

First-order derivative:

- would result in thick edges because the derivative is nonzero along a ramp.

Second-order derivative:

- would produce a double edge one pixel thick, separated by zeros

  => Enhances fine details much better than the first derivative, and are also easier to implement!

⇒ The **Laplacian** is the simplest *isotropic* second-order derivative operator.

*(**isotropic** means here that the operator behaves the same in all directions. The Laplacian does not favor any particular direction (horizontal, vertical, or diagonal) when computing the rate of change in intensity.)*

⇒ The Laplacian detects edges in all directions because it is isotropic.

Sharpening: The Laplacian filter

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

In discrete form, the Laplacian can be expressed in term of finite differences:

$$\nabla^2 f(x,y) = f(x+1,y) + f(x-1,y) +$$
$$+ f(x,y+1) + f(x,y-1) - 4f(x,y)$$

Sharpening: The Laplacian filter

Since derivatives are linear operators, the Laplacian is a linear operator and hence can be implemented as a convolution with a proper filter mask:

0	1	0
1	−4	1
0	1	0

This filter gives an isotropic result for increments of 90°

1	1	1
1	−8	1
1	1	1

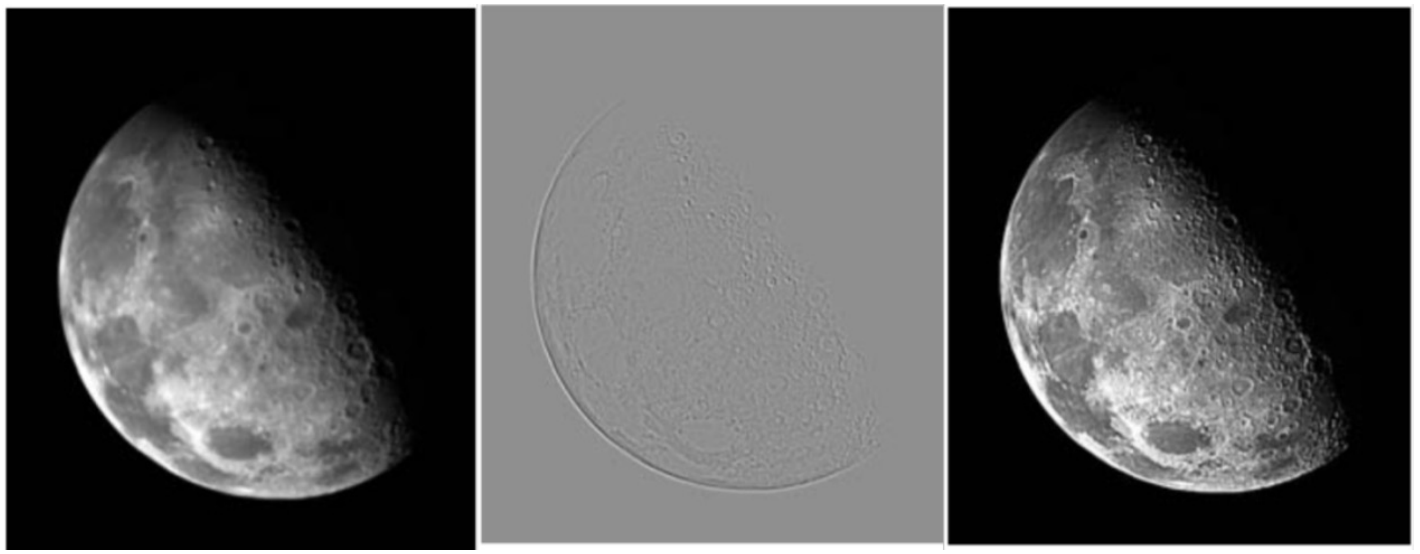
This filter gives an isotropic result for increments of 45°

The diagonal directions can be incorporated in the definition of the digital Laplacian by adding two more terms, one for each of the two diagonal directions.

Sharpening: The Laplacian filter

- The Laplacian operator highlights intensity discontinuities in an image and deemphasizes regions with slowly varying intensity levels.
- If we add the effect of the Laplacian operator to the original image, we are effectively sharpening the details while preserving background slowly-varying gradients.

$$g(x,y) = f(x,y) + c\left(\nabla^2 f(x,y)\right)$$



Spatial filtering: Non-linear filters

Order-statistic filters are non-linear spatial filters whose response is based on:

1. ordering the pixels contained in the image area encompassed by the filter
2. replacing the value of the center pixel with the value determined by the ranking result.

This category of filters is non-linear and cannot be performed as a convolution

Spatial filtering: Non-linear filters

Order-statistic filters

Median-filter: replaces the value of a pixel by the median of the intensity values in the neighborhood of that pixel (the original value of the pixel is included in the computation of the median)

Max-filter: Replaces the value of a pixel with the brightest intensity value in the neighborhood

Min-filter: Replaces the value of a pixel with the darkest intensity value in the neighborhood

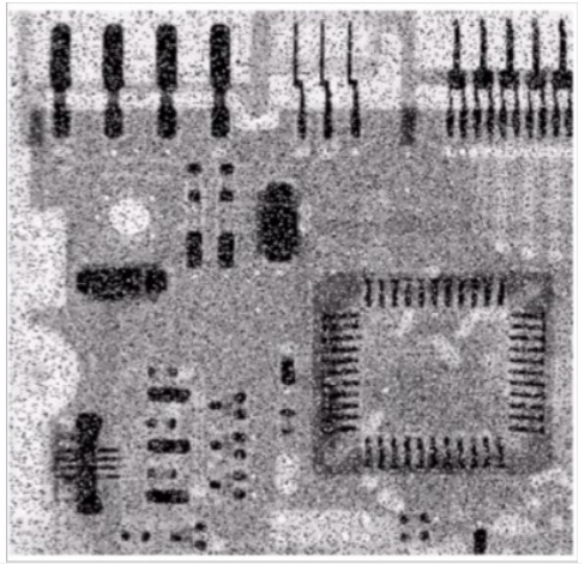
Spatial filtering: Non-linear filters

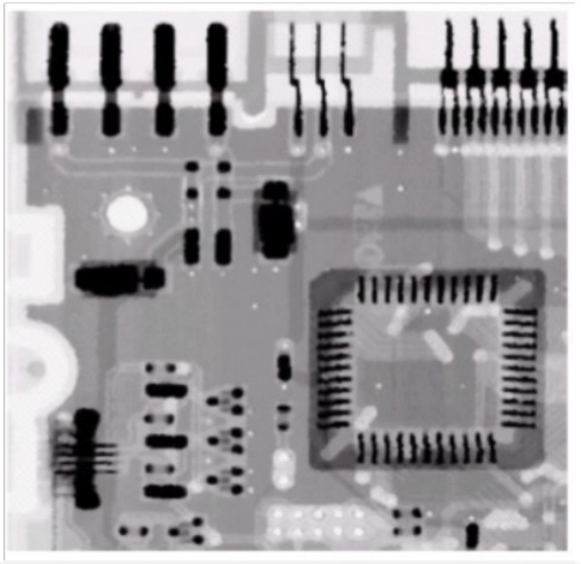
α-trimmed mean filter

- It uses a robust estimate of the mean (that does not take into account outliers):

  Eliminate the top and bottom α/2 values.

  Take the average of the remaining pixels.





Spatial filtering: Non-linear filters

Bilateral filtering

- It is a non-linear, edge-preserving smoothing technique used in image processing.
- Unlike traditional filters that blur edges, it considers both **spatial proximity** and **intensity similarity** when averaging pixel values.
- **A bilateral filter smooths an image while preserving edges.**
  It works by averaging nearby pixels, but it gives more importance to pixels that are both **spatially close** and **similar in intensity**.
- **Spatial Weighting:** Nearby pixels have a higher influence (Gaussian based on distance).
- **Intensity Weighting:** Pixels with similar intensity values have a higher influence (Gaussian based on intensity difference).

Spatial filtering: Non-linear filters

Bilateral filtering

- It considers both **spatial proximity** and **intensity similarity** when averaging pixel values.

$$I'(x) = \frac{1}{W_p} \sum_{x_i \in \mathcal{N}(x)} I(x_i) \cdot f_s(\|x_i - x\|) \cdot f_r(|I(x_i) - I(x)|)$$

$I'(x)$ is the filtered intensity at pixel x.

$I(x_i)$ is the intensity of a neighboring pixel x_i.

$\mathcal{N}(x)$ is the neighborhood around x.

$$f_s(\|x_i - x\|) = \exp\left(-\frac{\|x_i - x\|^2}{2\sigma_s^2}\right)$$

$$f_r(|I(x_i) - I(x)|) = \exp\left(-\frac{|I(x_i) - I(x)|^2}{2\sigma_r^2}\right)$$

$$W_p = \sum_{x_i \in \mathcal{N}(x)} f_s(\|x_i - x\|) \cdot f_r(|I(x_i) - I(x)|)$$

$$\left(W_P = \sum all\ weights\right)$$

Original

Gaussian $\sigma_s = 2$

Gaussian $\sigma_s = 2, \sigma_r = 10.$

Spatial filtering: Non-linear filters

Bilateral filtering

Why bilateral filtering is slow?

- Bilateral filtering is slow because the weights depend on both spatial distance and pixel intensity, so they must be recomputed for every pixel and every neighbor.
- The weights depend on:

  Distance → same as Gaussian

  Intensity difference → depends on pixel values => This term is the problem

- The filter is no longer linear or shift-invariant.

Spatial filtering

What will be seen in Lab 3:

- Other filters.
- Noise.
- Noise removal using filters.
- How to assess the effectiveness of the filters (qualitative/quantitative evaluations):
  - PSNR
  - SSIM

Correlation and convolution

Linear spatial filtering can be described in terms of correlation and convolution.

- **Correlation:**

The process of moving a filter mask over a signal (the image in our case) and computing the sum of products at each location.

- **Convolution:**

Similar to correlation but the filter mask is first rotated by 180°.

Correlation

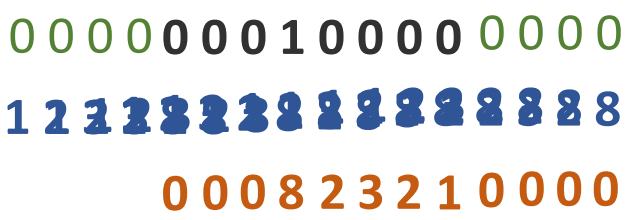
An example:

Suppose that we want to compute the correlation of the 1D signal:

f(x) = 0 0 0 1 0 0 0 0

with the mask:

w(x) = 1 2 3 2 8



Correlation vs. Convolution

- Correlation **is a function of displacement** of the filter. The first value of correlation corresponds to zero displacement, the second corresponds to one unit displacement, and so on
- Correlating a filter w with a function that contains all 0s and a single 1 yields a result that is a copy of w, but rotated by 180°
- Convolution works exactly the same way, but the filter is rotated by 180° before the shift operations.
- A fundamental property of convolution is that convolving a function with a unit impulse yields a copy of the mask at the location of the impulse.

2D Correlation/Convolution

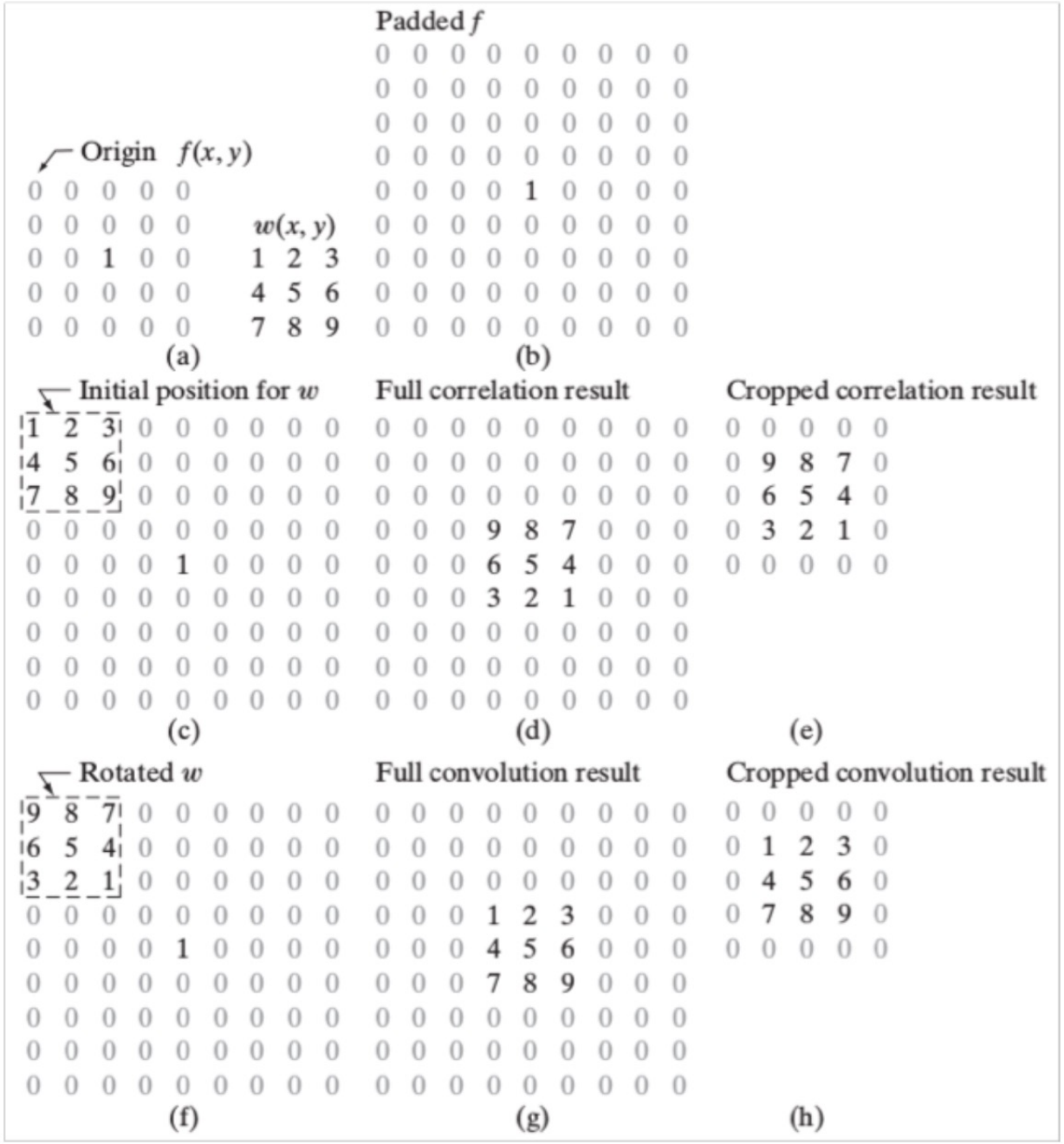
In case of 2D functions, like images, the correlation/convolution works in a similar manner:

- For a filter of size MxN we first pad the image with a minimum of:
  - M-1 rows at top and M-1 rows at bottom (filled with 0s)
  - N-1 cols at left and N-1 cols at right (filled with 0s)
- We shift the filter at each vertical and horizontal shift to perform the correlation/convolution operation:

Correlation: $$(w \otimes f)(x,y) = \sum_{s=-a}^{a}\sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$

Convolution: $$(w * f)(x,y) = \sum_{s=-a}^{a}\sum_{t=-b}^{b} w(s,t) f(x-s, y-t)$$

2D Correlation Convolution



Template Matching

Image *(f)*

Template *(w)*

- How do we locate the template in the image?

=> Minimize:

$$E[i,j] = \sum_{s=-a}^{a} \sum_{t=-b}^{b} [w(s,t) - f(i+s, j+t)]^2$$

Template Matching

$$E[i,j] = \sum_{s=-a}^{a} \sum_{t=-b}^{b} [w(s,t) - f(i+s, j+t)]^2$$

$$E[i,j] = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)^2 + f(i+s, j+t)^2 - 2\, w(s,t)\, f(i+s, j+t)$$

Minimizing $E[i,j]$ => Maximizing

$$\sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)\, f(i+s, j+t) = w \otimes f(i,j)$$

Filters as Templates

- Filters offer a natural mechanism for finding simple patterns.
- Filters respond most strongly to pattern elements that look like the filter.

Correlation as a Dot Product

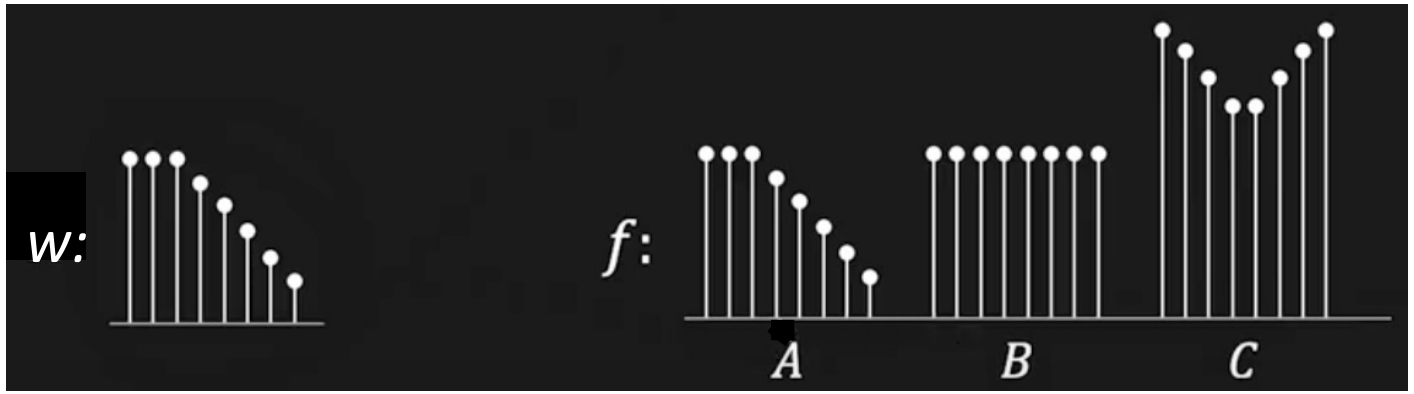
When performing a correlation (or a convolution):

- The response is obtained by associating image elements with filter kernel elements, multiplying the associated elements and summing
- It is the same process as a dot product.
- The dot product achieves its largest value when the vector representing the image is parallel to the vector representing the kernel.

$\Rightarrow$ A filter responds most strongly when it encounters an image pattern that looks like the filter.

$\Rightarrow$ But this dot product is a poor way to find patterns because the response might be large just because the image region is bright.

Correlation for Pattern Matching



$$\boldsymbol{R_{wf}[i,j] = w \otimes f(i,j) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)\, f(i+s, j+t)}$$

$$R_{wf}(C) > R_{wf}(B) > R_{wf}(A)$$

However, we need $\boldsymbol{R_{wf}(A)}$ to be the maximum.

Normalized Correlation for Pattern Matching

Solution: ***Normalizing the correlation***

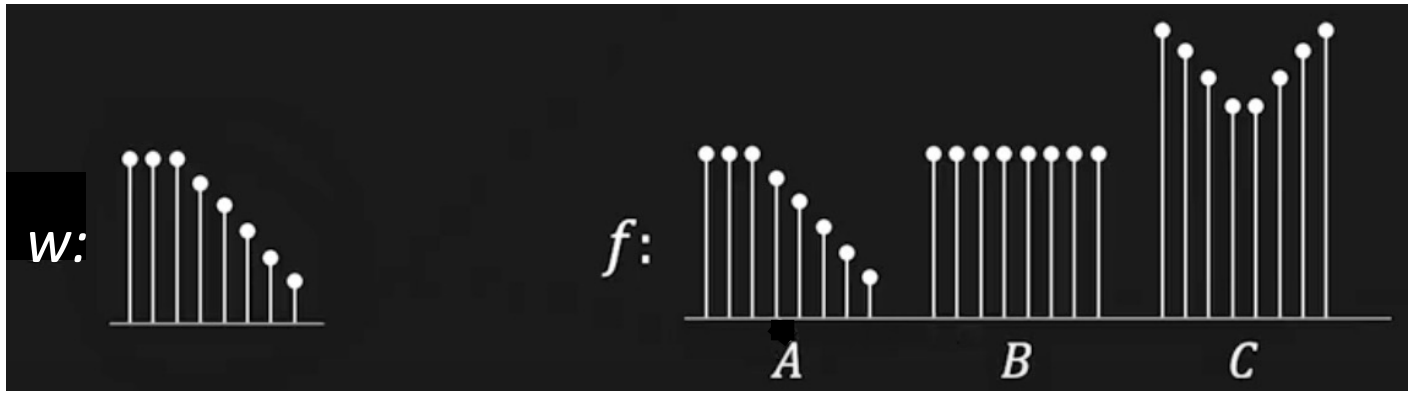
$$N_{wf}[i,j] = \frac{\sum_{s=-a}^{a}\sum_{t=-b}^{b} w(s,t)\, f(i+s, j+t)}{\sqrt{\sum_s \sum_t w(s,t)^2} \cdot \sqrt{\sum_s \sum_t f(i+s, i+t)^2}}$$

Energy of the template

Energy of the image area covered by the template

$\Rightarrow$ This make the correlation insensitive to brightness.

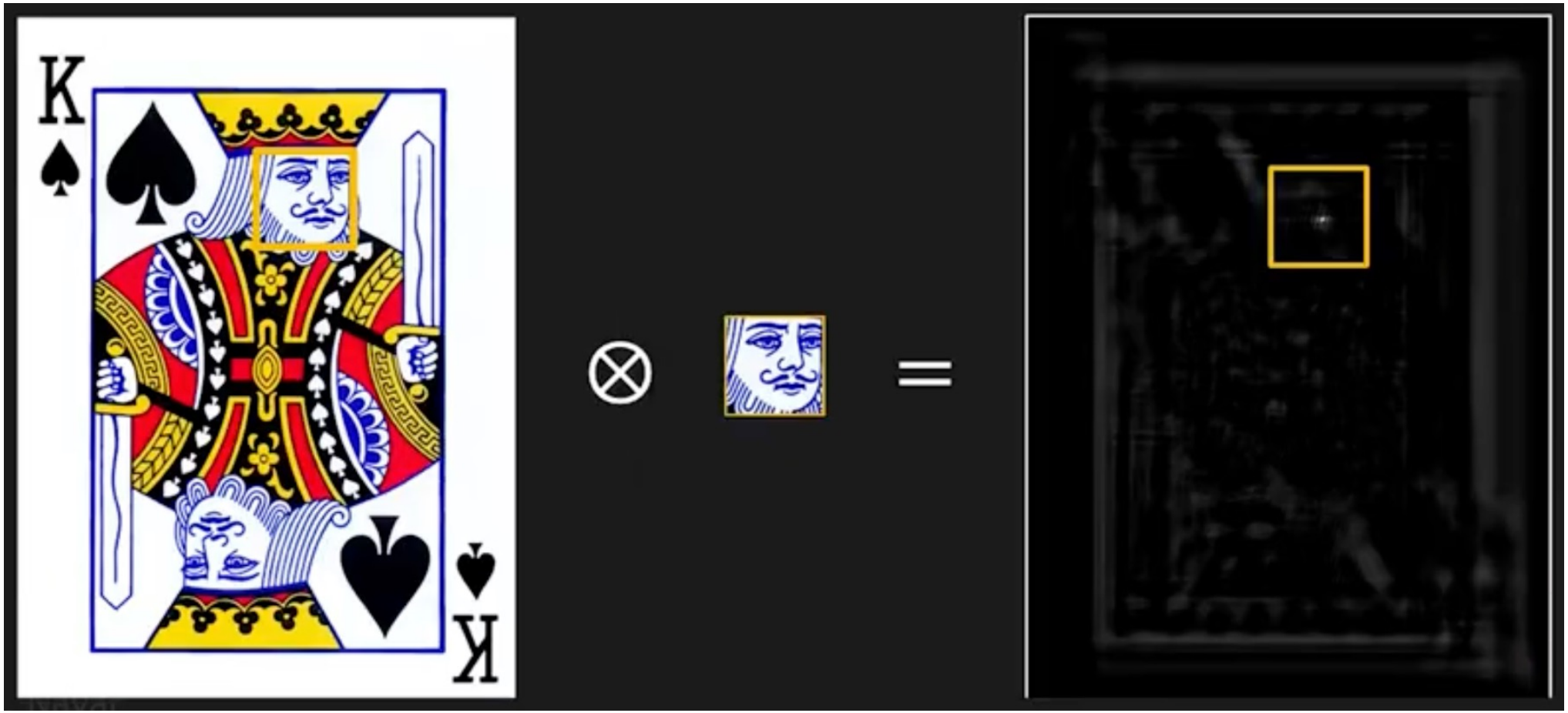
Normalized Correlation for Pattern Matching



$$N_{wf}[i,j] = \frac{\sum_{s=-a}^{a}\sum_{t=-b}^{b} w(s,t)\, f(i+s, j+t)}{\sqrt{\sum_s \sum_t w(s,t)^2}\,.\sqrt{\sum_s \sum_t f(i+s, i+t)^2}}$$

$$N_{wf}A > N_{wf}(B) > N_{wf}(C)$$

Normalized Correlation for Pattern Matching



Normalized Correlation for Pattern Matching

$\otimes$

=